EXPERIMENT 7: Spelling Correction using Edit Distance and Bigram Model

AIM:

To implement an automatic spelling correction system using Levenshtein Edit Distance for candidate generation and a Bigram Language Model to select the most contextually appropriate correction.

PROCEDURE:

1. Import numpy, collections (defaultdict, Counter).
2. Define corpus text for training.
3. Compute unigram and bigram counts from the corpus.
4. Build vocabulary.
5. Implement edit_distance(w1, w2) using dynamic programming.
6. Define generate_candidates(word): return all vocabulary words within edit distance

≤ 2.

7. Define bigram_prob(prev, word): compute Laplace-smoothed bigram probability.
8. Define correct_sentence(sentence): for each word, generate candidates and choose the one with highest bigram probability given the previous word.
9. Test with sentence "this corpus hep in corection".
10. Print Original and Corrected sentences.

CODE:

```
import numpy as np
from collections import defaultdict, Counter


corpus = """
this is a sample corpus for spelling correction this
corpus helps in correcting spelling mistakes we use
language models for correction
"""


words = corpus.lower().split() unigrams
= Counter(words) bigrams =
defaultdict(int) for i in
range(len(words)-1):
bigrams[(words[i], words[i+1])] += 1


vocab = set(words)


def edit_distance(w1, w2):     dp =
np.zeros((len(w1)+1, len(w2)+1))     for i
in range(len(w1)+1): dp[i][0] = i     for j
in range(len(w2)+1): dp[0][j] = j
```

```
    for i in range(1, len(w1)+1):
for j in range(1, len(w2)+1):
            cost = 0 if w1[i-1] == w2[j-1] else 1             dp[i][j] =
min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+cost)     return dp[-1][-1]


def generate_candidates(word):
candidates = []     for v in vocab:
if edit_distance(word, v) <= 2:
            candidates.append(v)
    return candidates if candidates else [word]


def bigram_prob(prev, word):     return (bigrams[(prev, word)] + 1) /
(unigrams[prev] + len(vocab))


def correct_sentence(sentence):
tokens = sentence.lower().split()
corrected = [tokens[0]]     for i in
range(1, len(tokens)):
        prev = corrected[-1]         word =
tokens[i]         candidates =
generate_candidates(word)         best_word =
word         best_score = -1         for c in
candidates:
            score = bigram_prob(prev, c)
if score > best_score:
best_score = score
best_word = c
corrected.append(best_word)     return "
".join(corrected)
```

```
sentence = "this corpus hep in corection" print("Original:",
sentence)
print("Corrected:", correct_sentence(sentence))
```

OUTPUT:

```
Original: this corpus hep in corection
Corrected: this corpus helps in correction
```

RESULT:

The spelling correction system successfully corrected 'hep' to 'helps' and 'corection' to 'correction'. The combination of edit distance (for candidate generation) and bigram probability (for context-aware selection) produced accurate corrections. The system correctly leverages contextual information to choose the most appropriate candidate.